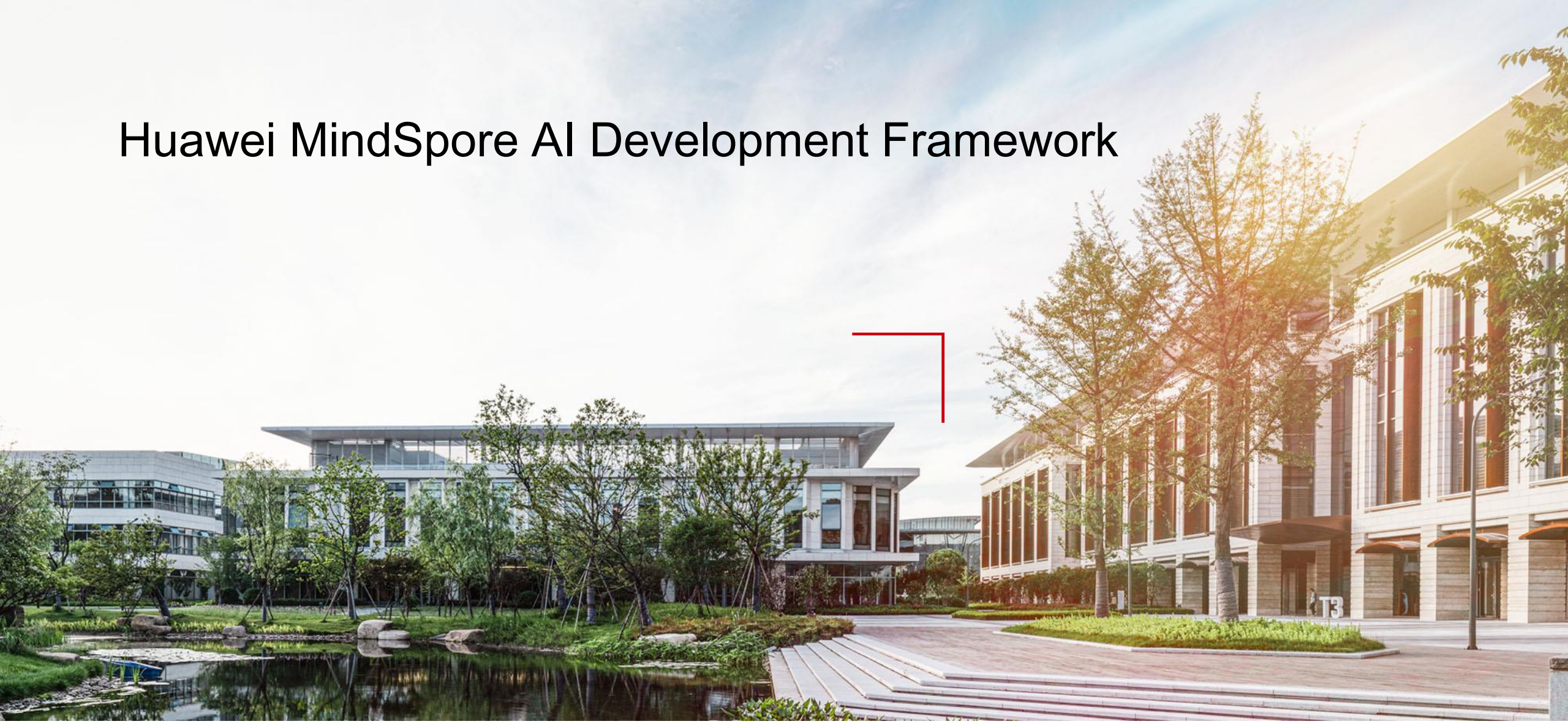
Huawei MindSpore AI Development Framework



HUAWEI

Foreword

- This chapter introduces the structure, design concept, and features of MindSpore based on the issues and difficulties facing by the AI computing framework, and describes the development and application process in MindSpore.

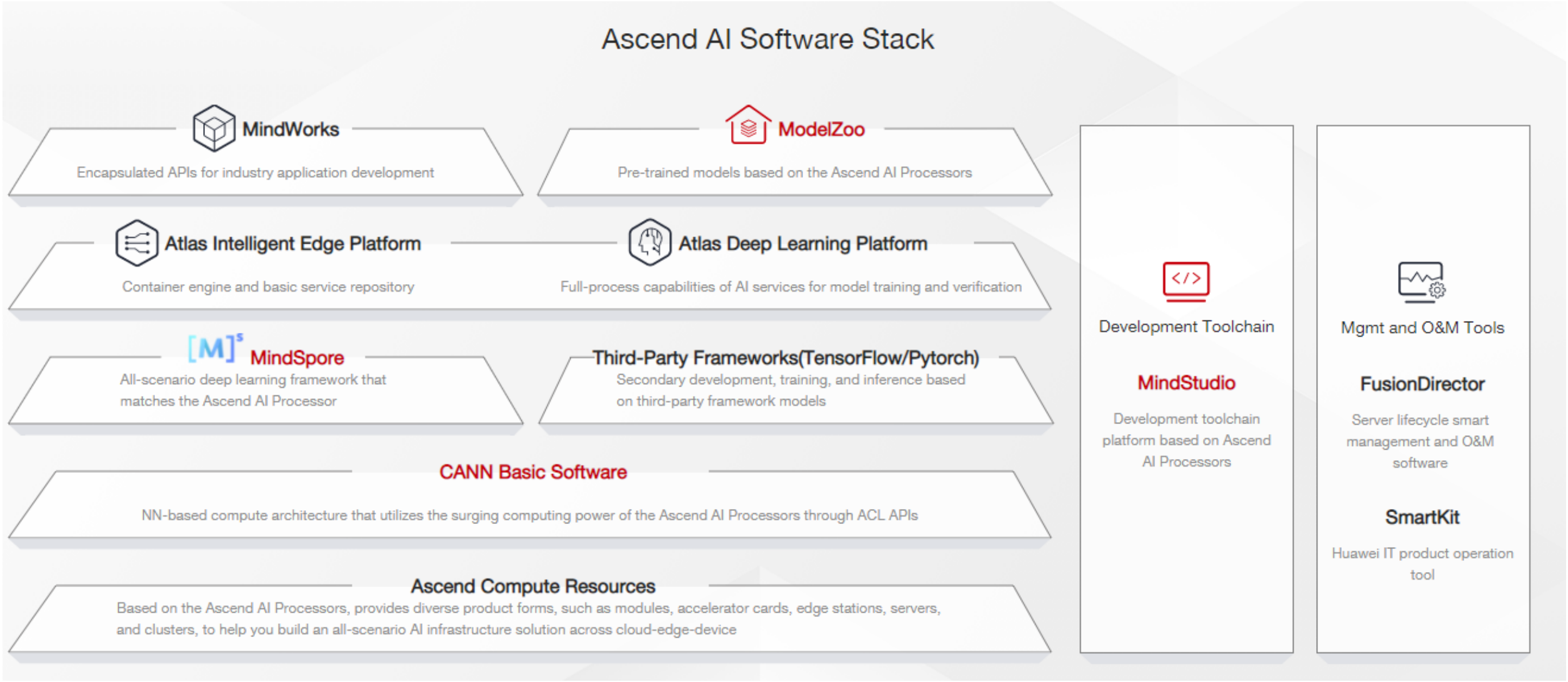
Objectives

Upon completion of this course, you will be able to:

- Learn what MindSpore is
- Understand the framework of MindSpore
- Understand the design concept of MindSpore
- Learn features of MindSpore
- Grasp the environment setup process and development cases

Contents

Ascend AI Software Stack



How will deep learning develop?

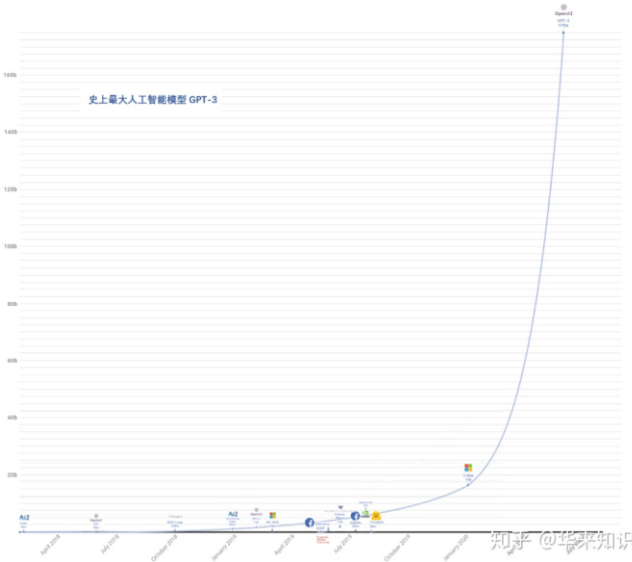
The demands for computing power will keep increasing

15B

340M

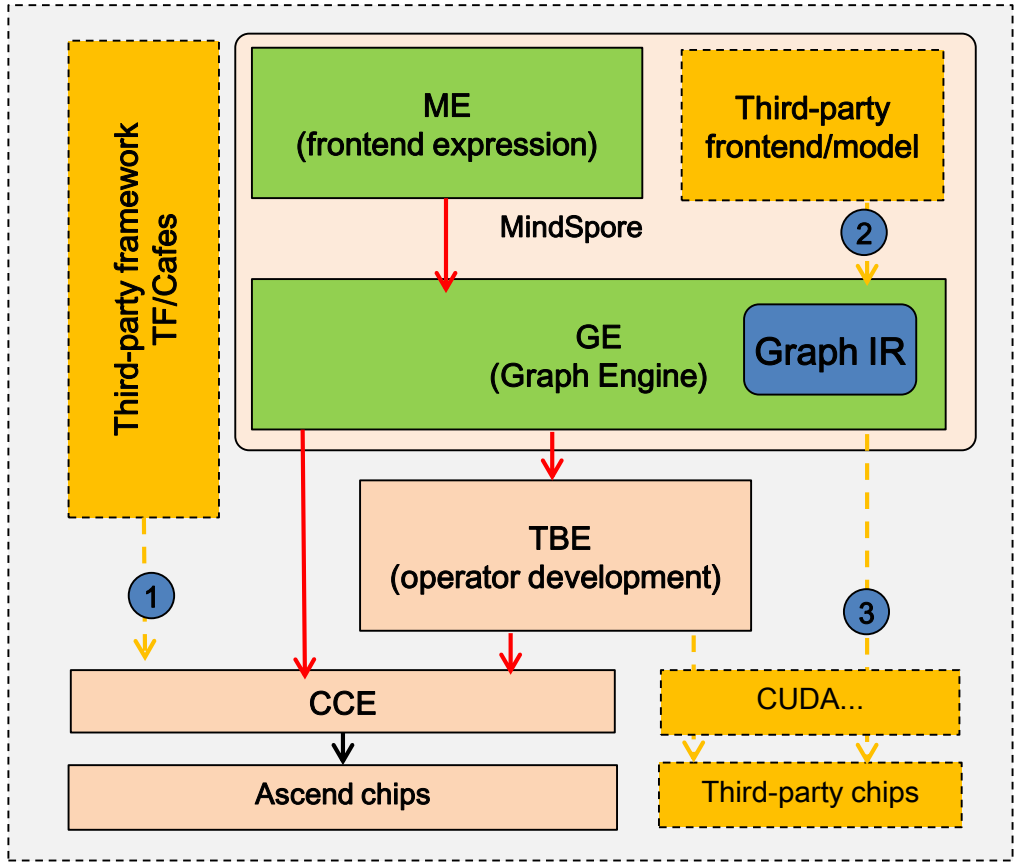
90M

ELMO Bert GPT-3



HUAWEI

Architecture: Easy Development and Efficient Execution



ME (Mind Expression): interface layer (Python)

Usability: automatic differential programming and original mathematical expression

- Auto diff: operator-level automatic differential
- Auto parallel: automatic parallelism
- Auto tensor: automatic generation of operators
- Semi-auto labeling: semi-automatic data labeling

GE (Graph Engine): graph compilation and execution layer

High performance: software/hardware co-optimization, and full-scenario application

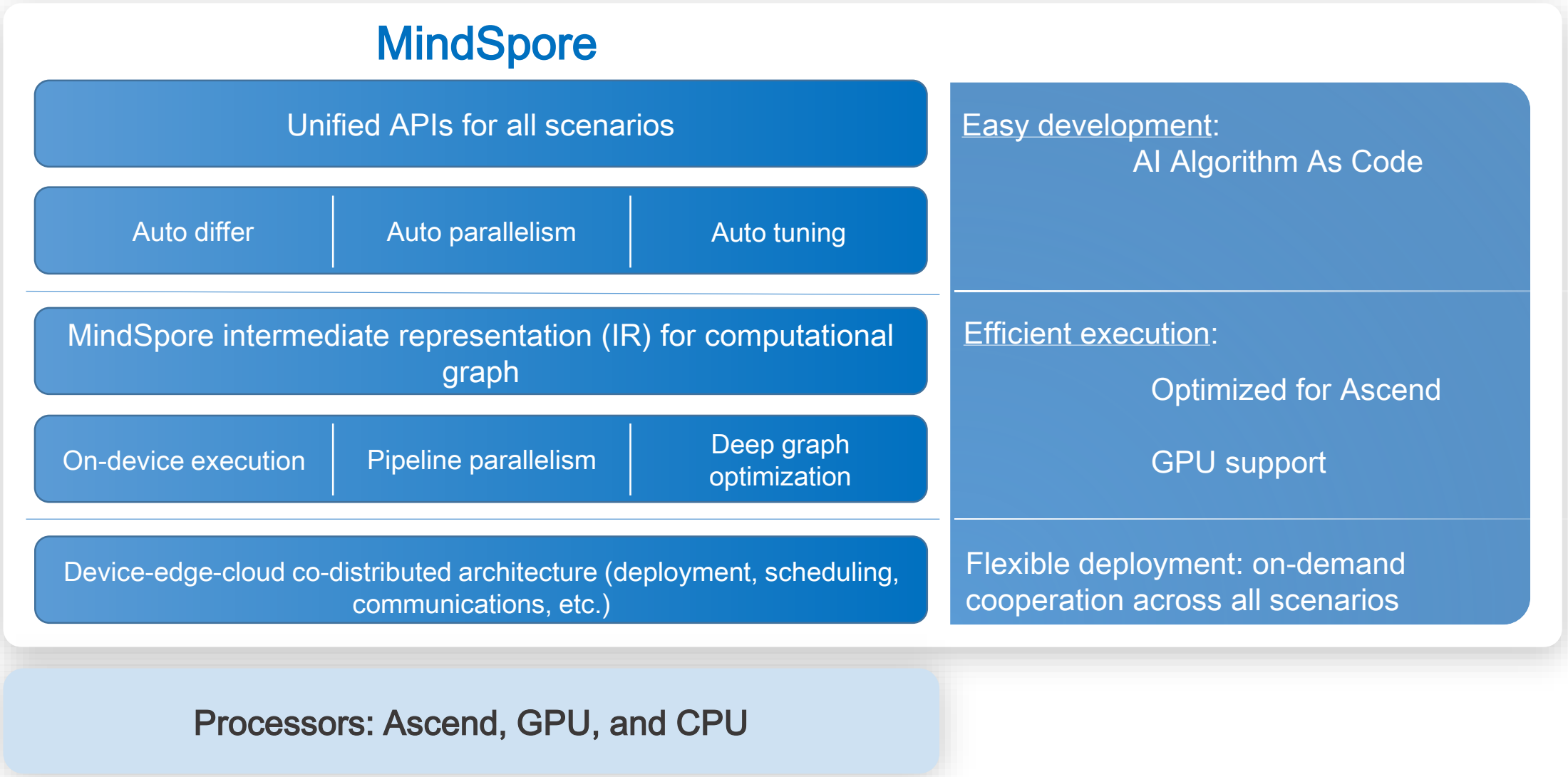
- Cross-layer memory overcommitment
- Deep graph optimization
- On-device execution
- Device-edge-cloud synergy (including online compilation)

① Equivalent to open-source frameworks in the industry, MindSpore preferentially serves self-developed chips and cloud services.

② It supports upward interconnection with third-party frameworks and can interconnect with third-party ecosystems through Graph IR, including training frontends and inference models. Developers can expand the capability of MindSpore.

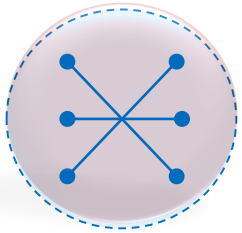
③ It also supports interconnection with third-party chips and helps developers increase MindSpore application scenarios and expand the AI ecosystem.

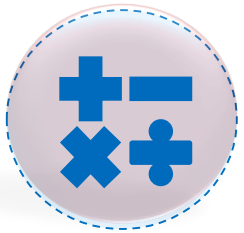
Overall Solution: Core Architecture

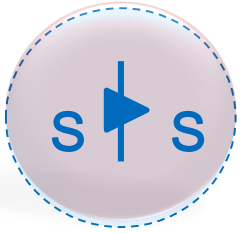


MindSpore Design: Auto Differ

Technical path of automatic differential







Graph: TensorFlow

- Non-Python programming based on graphs
- Complex representation of control flows and higher-order derivatives

Operator overloading: PyTorch

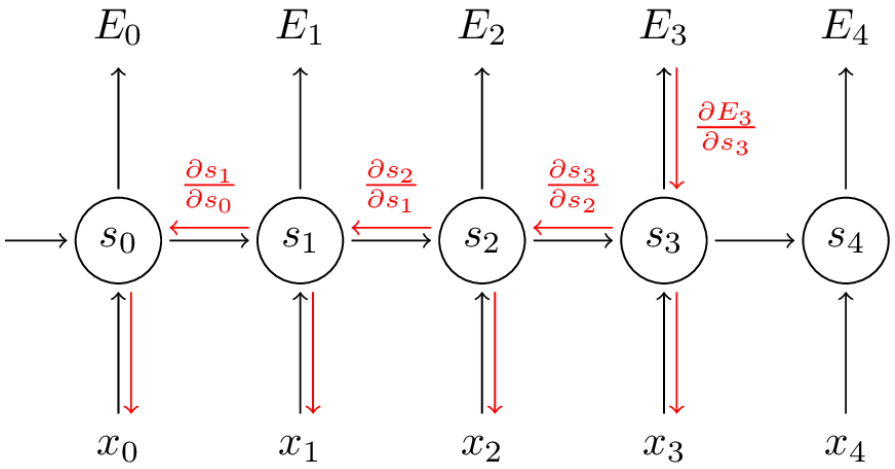
- Runtime overhead
- Backward process performance is difficult to optimize.

Source code transfer: MindSpore

- Python APIs for higher efficiency
- IR-based compilation optimization for better performance

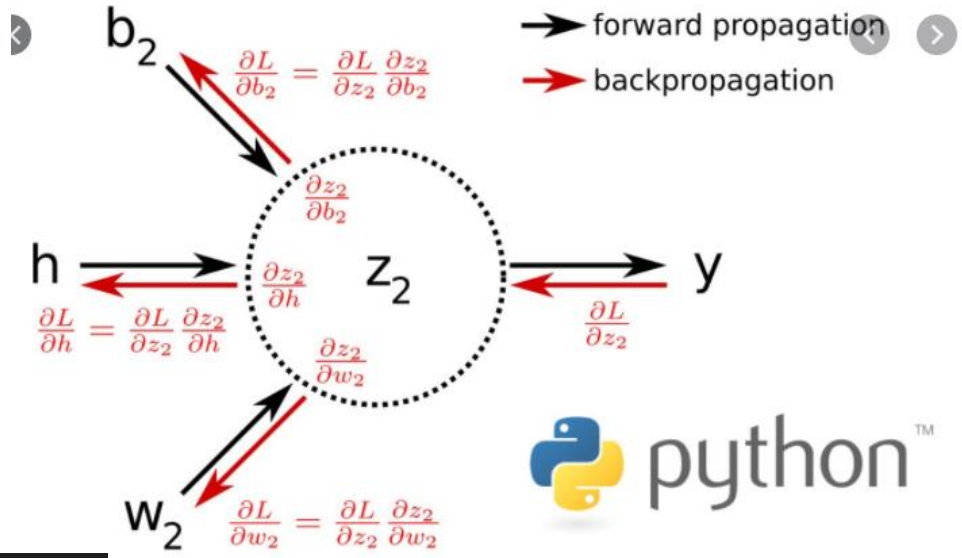
Example of Differentitation

Backpropagation Through Time (BPTT)



$$\frac{\partial E_3}{\partial W} = \sum_{k=0}^{3} \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial s_k} \frac{\partial s_k}{\partial W}$$

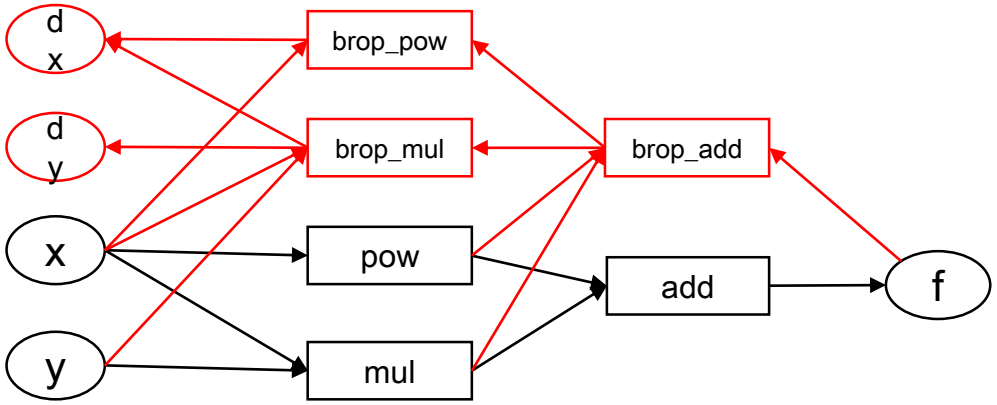
Single neuron back propagation




```
def tanh(x):
    return (1.0 - np.exp(-x))  / (1.0 + np.exp(-x))

x = np.linspace(-7, 7, 200)
plt.plot(x, tanh(x),
         x, grad(tanh)(x),                                  # first  derivative
         x, grad(grad(tanh))(x),                            # second derivative
         x, grad(grad(grad(tanh)))(x),                      # third  derivative
         x, grad(grad(grad(grad(tanh))))(x),                # fourth derivative
         x, grad(grad(grad(grad(grad(tanh)))))(x),          # fifth  derivative
         x, grad(grad(grad(grad(grad(grad(tanh))))))(x))    # sixth  derivative
```


HUAWEI

Graph-Based AD

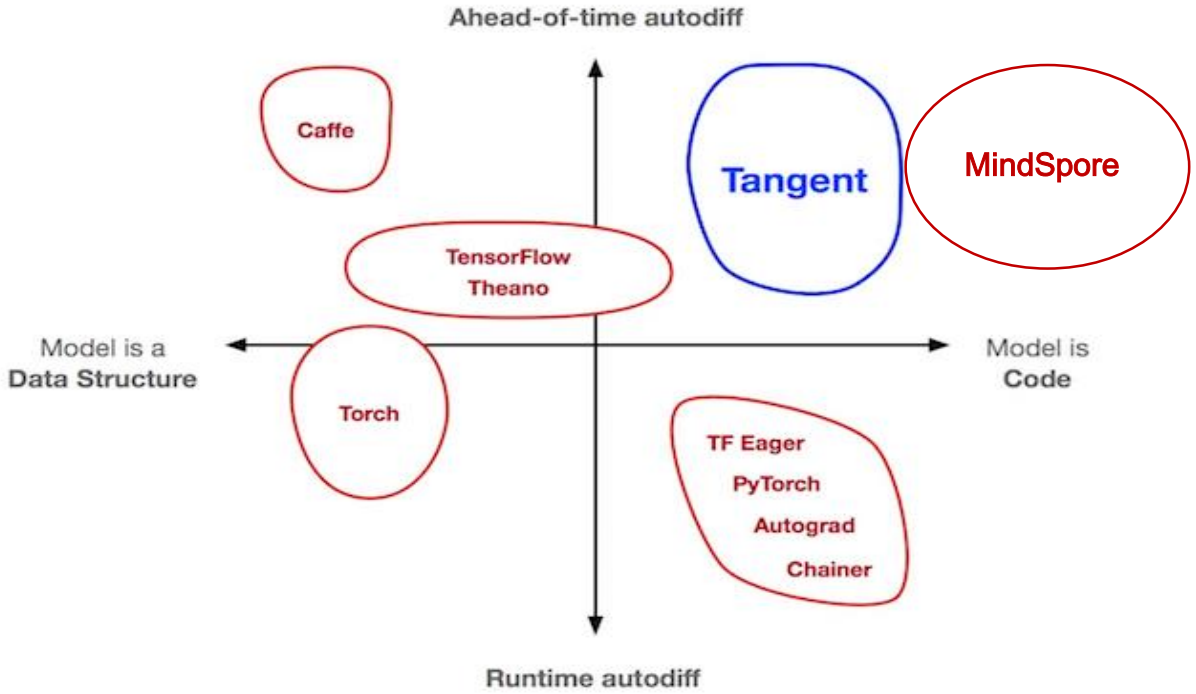


Pros：
1. Static graph：easy to optimize、parallel

Cons：
1. Extra learning
2. Easy to cause the problem of graph expansion

Source Code Transformation

If you want both graph-based performance and oo-based programming experience, is there any way to accomplish ?



```
import tangent
df = tangent.grad(f)
```

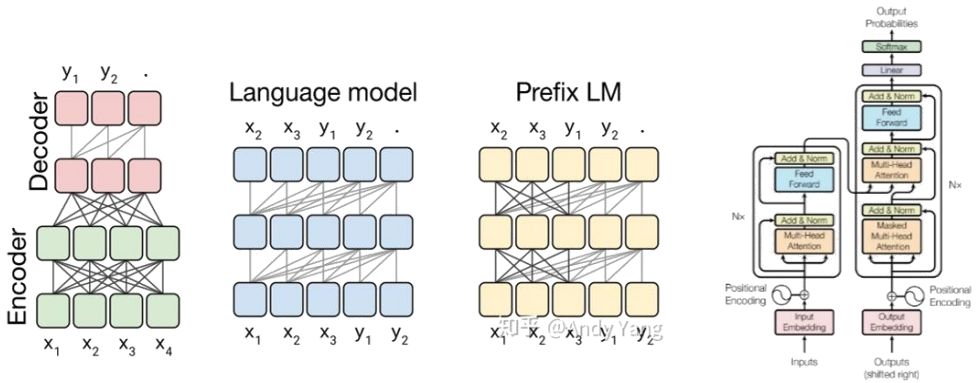
Source Code Transformation with Tangent

```
def f(x):
  a = x * x
  b = x * a
  c = a + b
  return c
```

Auto Parallelism

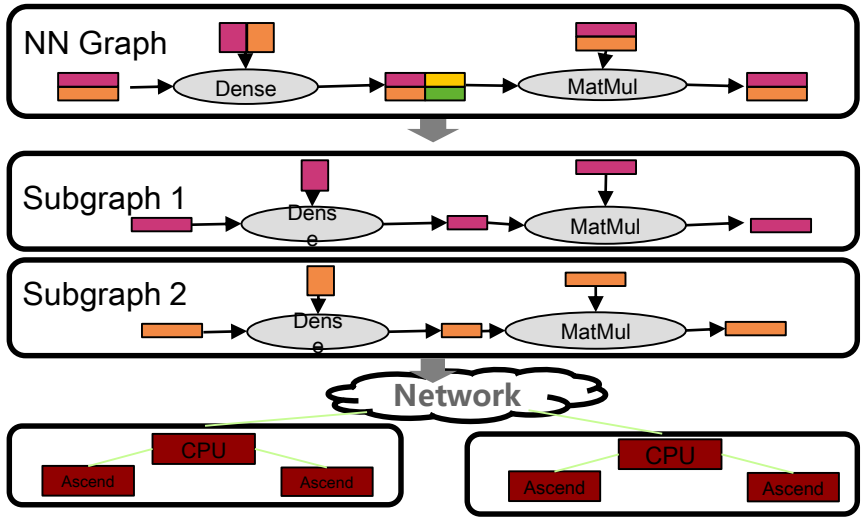
Challenges

Ultra-large models realize efficient distributed training:
As NLP-domain models swell, the memory overhead for training ultra-large models such as Bert (340M)/GPT-2(1542M) has exceeded the capacity of a single card. Therefore, the models need to be split into multiple cards before execution.
Manual model parallelism is **used currently**. Model segmentation needs to be designed and the cluster topology needs to be understood. The development is extremely challenging. The performance is lackluster and can be hardly optimized.



Key Technologies

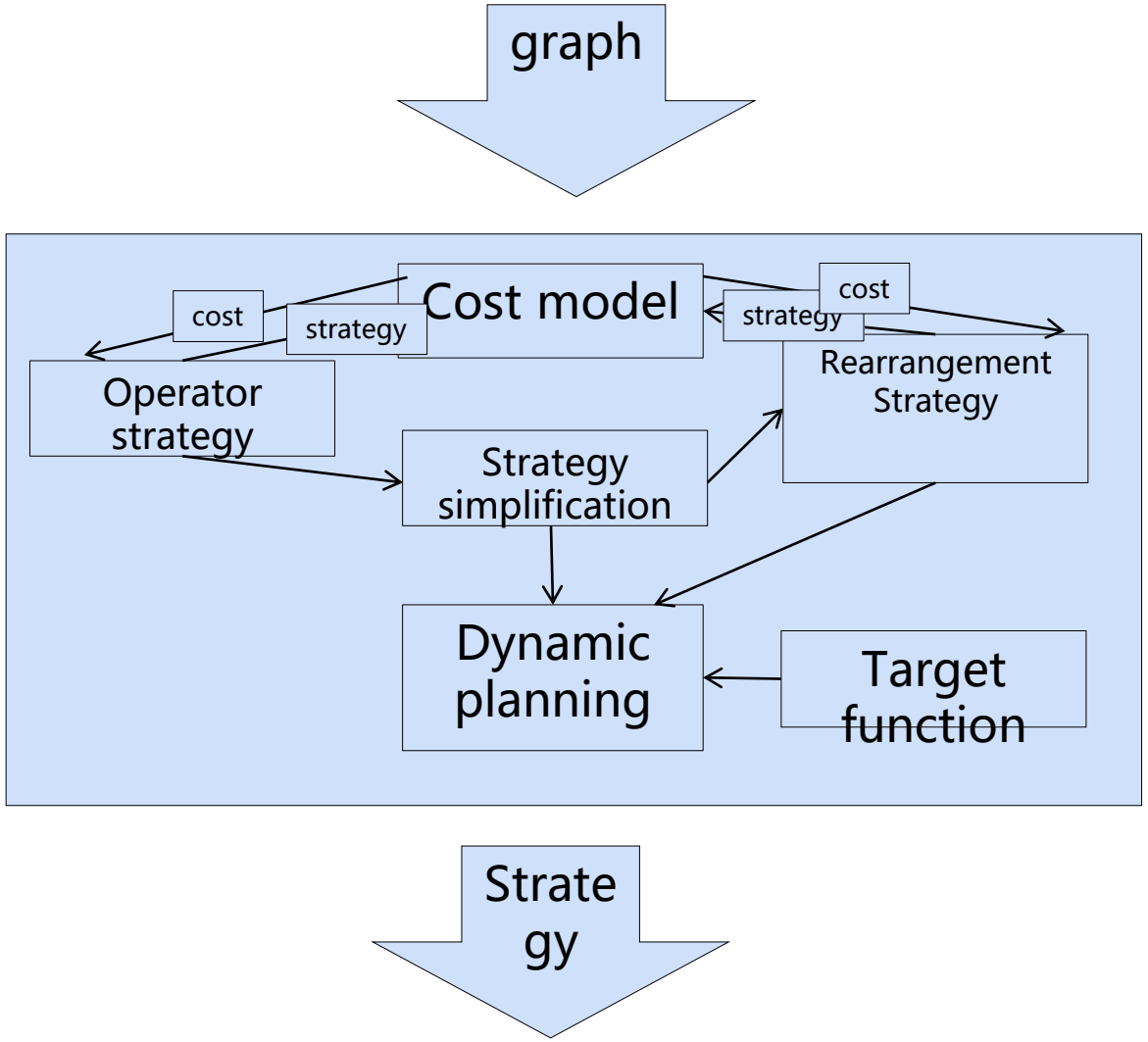
Automatic graph segmentation: It can segment the entire graph based on the input and output data dimensions of the operator, and integrate the data and model parallelism. Cluster topology awareness scheduling: It can perceive the cluster topology, schedule subgraphs automatically, and minimize the communication overhead.



Effect: Realize model parallelism based on the existing single-node code logic, improving the development efficiency tenfold compared with manual parallelism.

Segmentation strategy search

Highlight2：High effiency searching algorithm

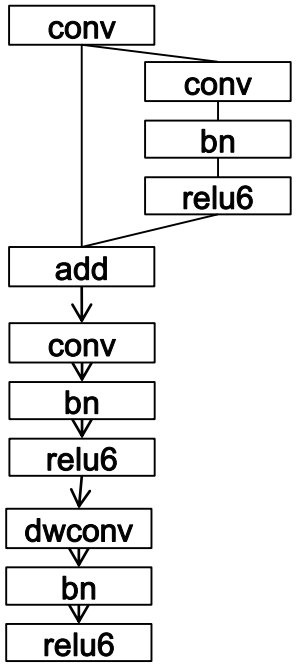


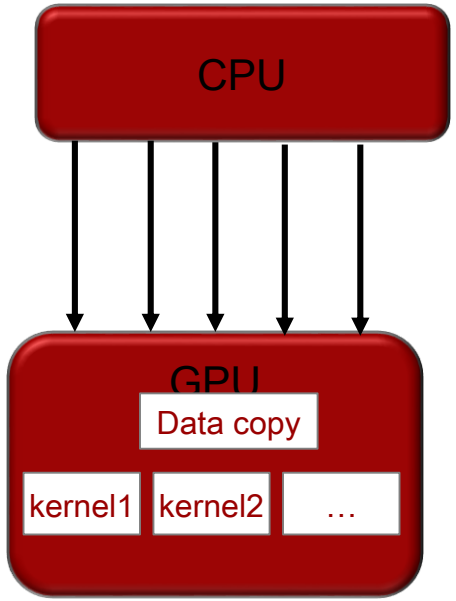
On-Device Execution (1)

Challenges

Challenges for model execution with supreme chip computing power:

Memory wall, high interaction overhead, and data supply difficulty. Partial operations are performed on the host, while the others are performed on the device. The interaction overhead is much greater than the execution overhead, resulting in the low accelerator usage.

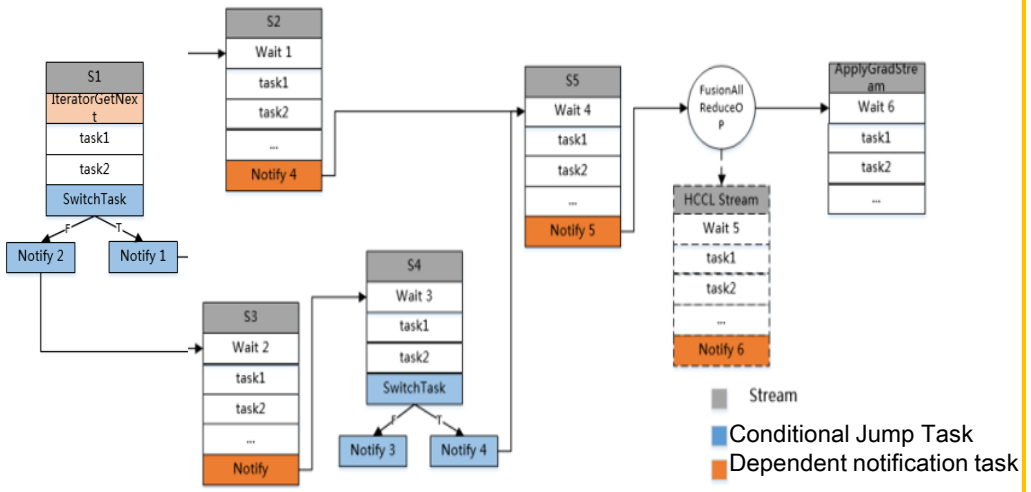




Large data interaction overhead and difficult data supply

Key Technologies

Chip-oriented deep graph optimization reduces the synchronization waiting time and maximizes the parallelism of data, computing, and communication. Data pre-processing and computation are integrated into the Ascend chip:



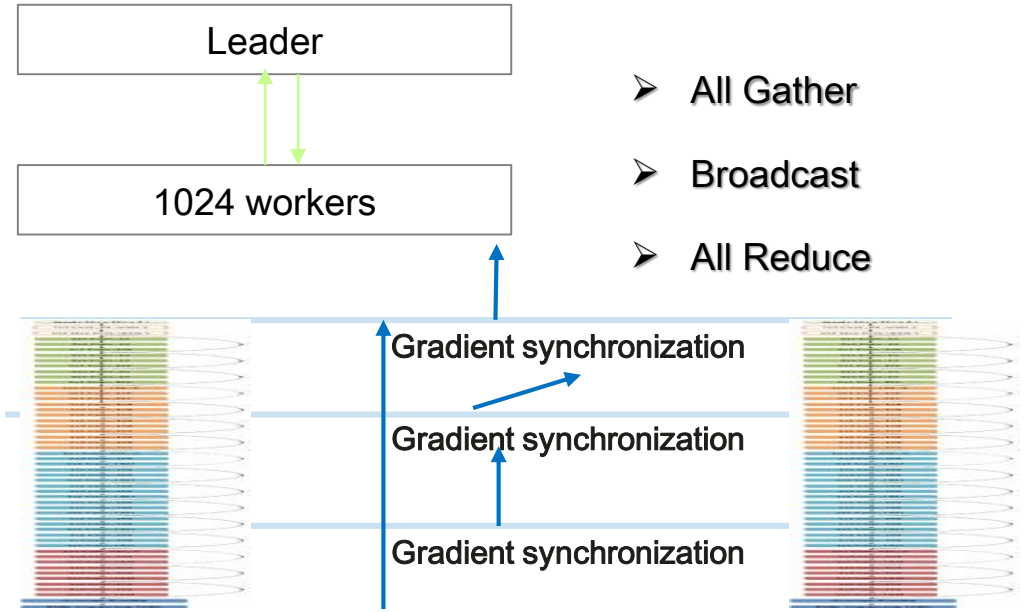
Effect: Elevate the training performance tenfold compared with the on-host graph scheduling.

HUAWEI

On-Device Execution (2)

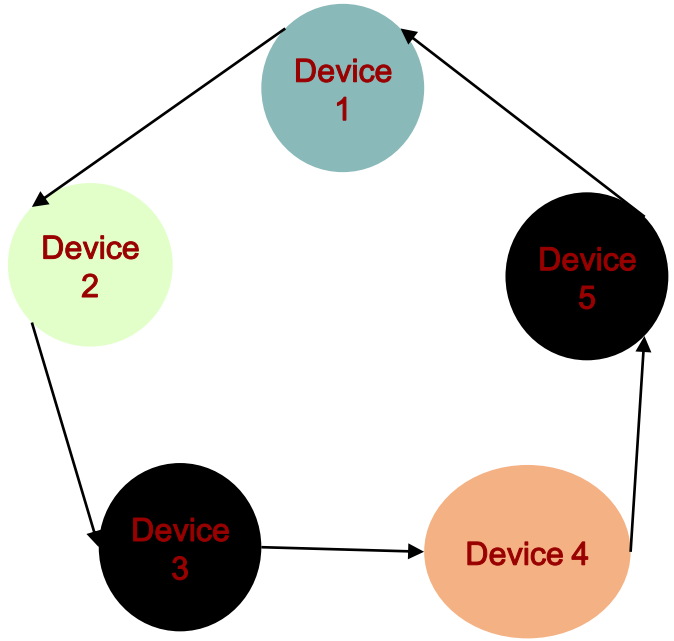
Challenges

Challenges for distributed gradient aggregation with supreme chip computing power:
the synchronization overhead of central control and the communication overhead of frequent synchronization of ResNet50 under the single iteration of 20 ms; the traditional method can only complete All Reduce after three times of synchronization, while the data-driven method can autonomously perform All Reduce without causing control overhead.



Key Technologies

The optimization of the **adaptive graph segmentation driven by gradient data** can realize decentralized All Reduce and synchronize gradient aggregation, boosting computing and communication efficiency.



Effect: a smearing overhead of less than 2 ms

Distributed Device-Edge-Cloud Synergy Architecture

Challenges

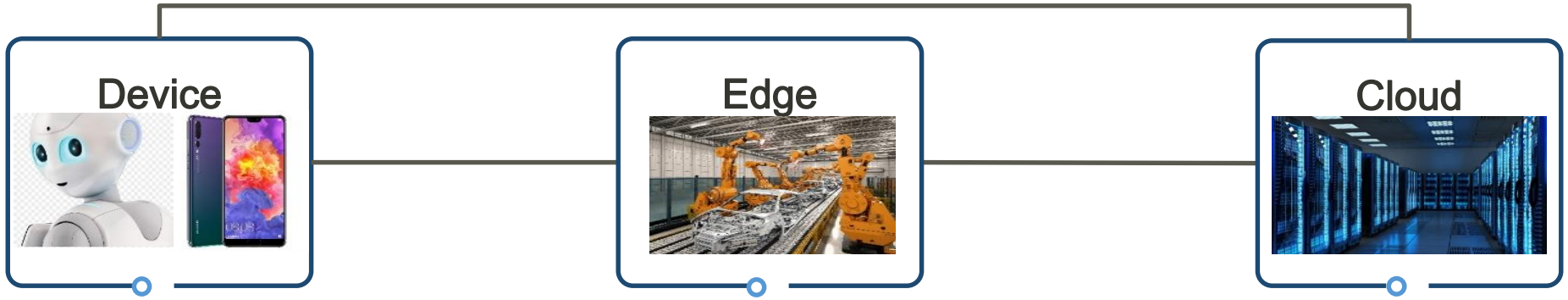
The diversity of hardware architectures leads to full-scenario deployment differences and performance uncertainties. The separation of training and inference leads to isolation of models.

Key Technologies

- **Unified model IR** delivers a consistent deployment experience.
- **The graph optimization technology featuring software and hardware collaboration** bridges different scenarios.
- Device-cloud Synergy Federal Meta Learning breaks the device-cloud boundary and updates the multi-device collaboration model in real time.

Effect: consistent model deployment performance across all scenarios thanks to the unified architecture, and improved precision of personalized models

On-demand collaboration in all scenarios and consistent development experience



HUAWEI

Contents

AI Computing Framework: Challenges

Industry Challenges

A huge gap between

industry research and all-scenario AI application

- High entry barriers
- High execution cost
- Long deployment duration

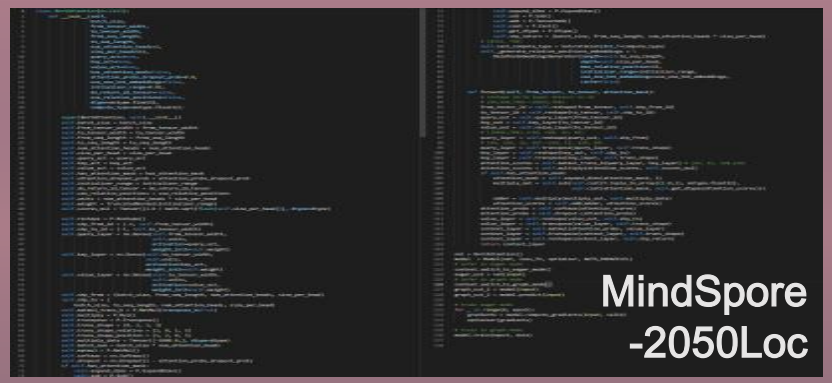
Technological Innovation

MindSpore facilitates inclusive AI across applications

- New programming mode
- New execution mode
- New collaboration mode

New Programming Paradigm

Algorithm scientist

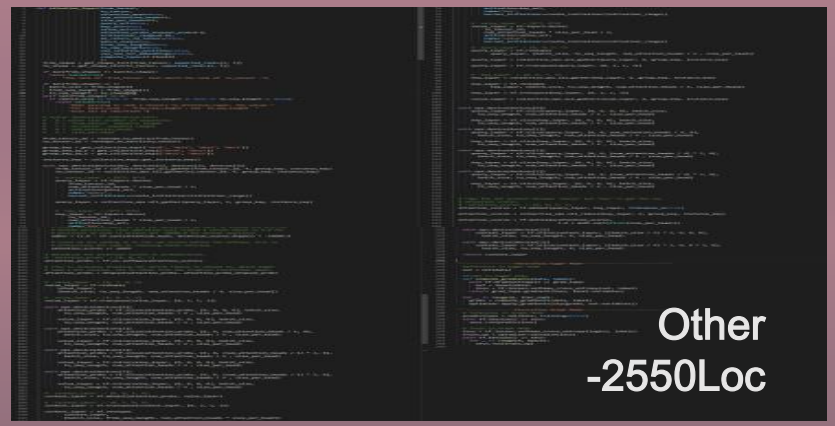


One-line
Automatic parallelism

Efficient automatic differential

One-line debug-mode switch

Algorithm scientist
+
Experienced system developer



NLP Model: Transformer

HUAWEI

Code Example

TensorFlow code snippet: XX lines, manual parallelism

```
import tensorflow as tf

model() {
    with tf.device("/device:0")
        token_type_table = tf.get_variable(
            name=token_type_embedding_name,
        shape=[token_type_vocab_size, width],
        initializer=create_initializer(initializer_range))
        flat_token_type_ids = tf.reshape(token_type_ids, [-1])
        one_hot_ids = tf.one_hot(flat_token_type_ids, depth=token_type_vocab_size)
        token_type_embeddings = tf.matmul(one_hot_ids, token_type_table)

    with tf.device("/device:1")
        query_layer = tf.layers.dense(
            from_tensor_2d,
            num_attention_heads * size_per_head,
            activation=query_act,
            name="query",
            kernel_initializer=create_initializer(initializer_range))

    with tf.device("/device:2")
        key_layer = tf.layers.dense(
            to_tensor_2d,
            num_attention_heads * size_per_head,
            activation=key_act,
            name="key",
            kernel_initializer=create_initializer(initializer_range))
```

MindSpore code snippet: two lines, automatic parallelism

```
class DenseMatMulNet(nn.Cell):
    def __init__(self):
        super(DenseMutMulNet, self).__init__()
        self.matmul1 = ops.MatMul.set_strategy({[4, 1], [1, 1]})
        self.matmul2 = ops.MatMul.set_strategy({[1, 1], [1, 4]})
    def construct(self, x, w, v):
        y = self.matmul1(x, w)
        z = self.matmul2(y, v)
        return s
```

Typical scenarios: ReID

New Execution Mode (1)

Execution Challenges

Complex AI computing and diverse computing units

1. CPU cores, cubes, and vectors
2. Scalar, vector, and tensor computing
3. Mixed precision computing
4. Dense matrix and sparse matrix computing

Multi-device execution: High cost of parallel control

Performance cannot linearly increase as the node quantity increases.

On-device execution

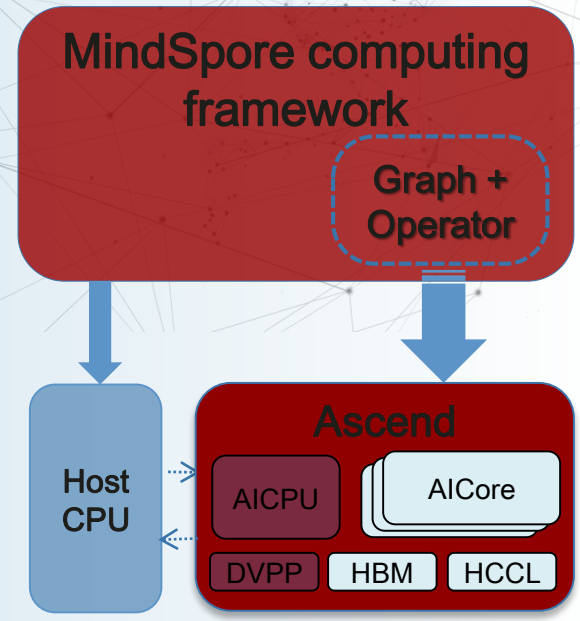
Offloads graphs to devices, maximizing the computing power of Ascend

Framework optimization

- Pipeline parallelism
- Cross-layer memory overcommitment

Soft/hardware co-optimization

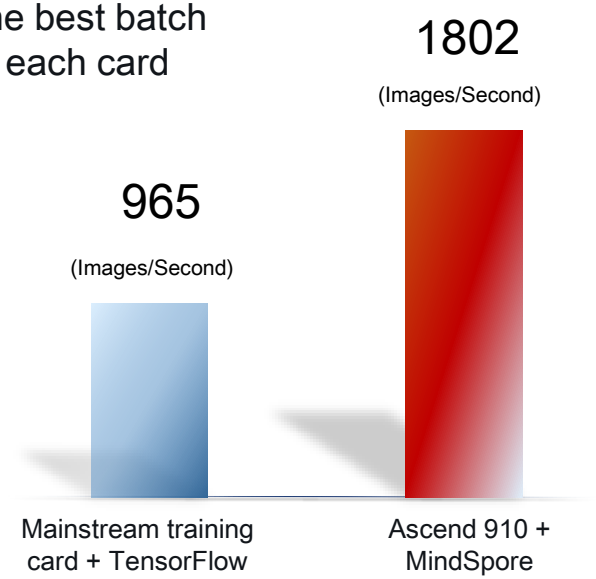
- On-device execution
- Deep graph optimization



HUAWEI

New Execution Mode (2)

- ResNet 50 V1.5
- ImageNet 2012
- With the best batch size of each card



Performance of ResNet-50 is doubled.
Single iteration:
58 ms (other frameworks+V100) v.s. about **22 ms** (MindSpore) (ResNet50+ImageNet, single-server, eight-device, batch size=32)

Detecting objects in 60 ms

Tracking objects in 5 ms

Multi-object real-time recognition
MindSpore-based mobile deployment, a smooth experience of multi-object detection

New Collaboration Mode

Deployment Challenge

v.s

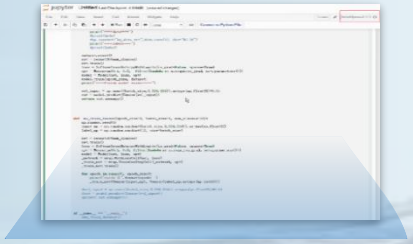
.

- Varied requirements, objectives, and constraints for device, edge, and cloud application scenarios

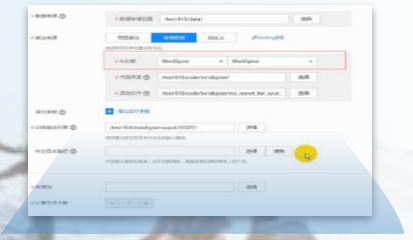
v.s.

- Different hardware precision and speed

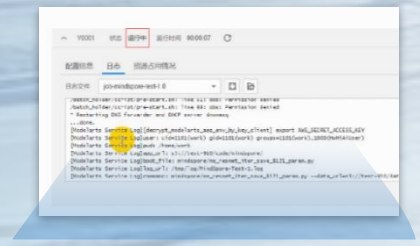
Unified development; flexible deployment; on-demand collaboration, and high security and reliability



Development



Deployment



Execution

Saving model

Unified development and flexible deployment

High Performance

AI computing challenges

Complex computing

Scalar, vector, and tensor computing

Mixed precision computing

Parallelism between gradient aggregation and mini-batch computing

Diverse computing units / processors

CPUs, GPUs, and Ascend processors

Scalar, vector, and tensor

MindSpore

Framework optimization

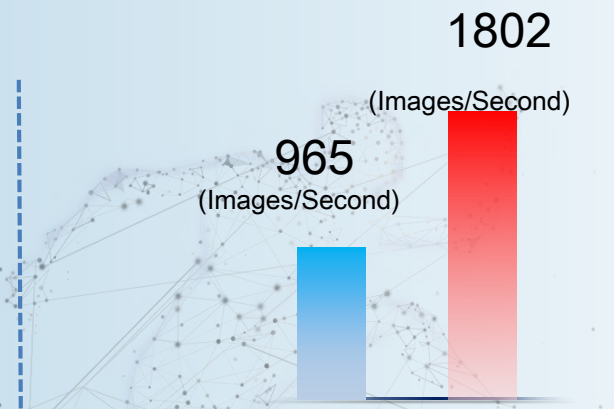
Pipeline parallelism

Cross-layer memory overcommitment

Software + hardware co-optimization

On-device execution

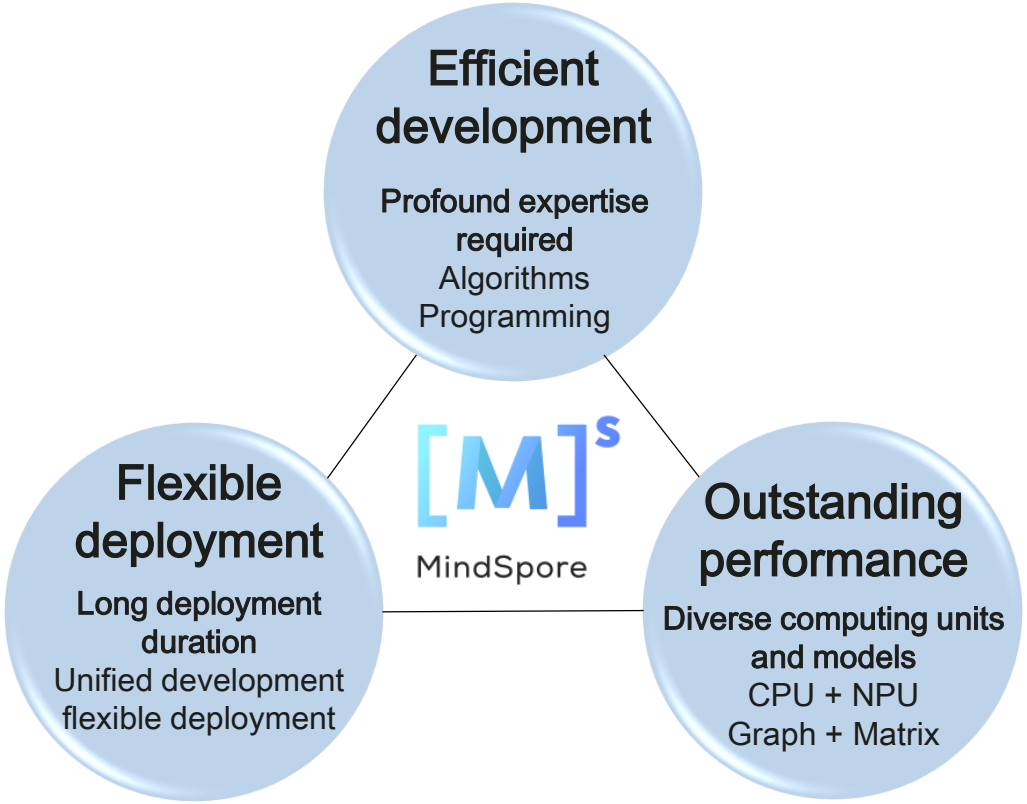
Deep graph optimization



- ResNet 50 V1.5
- ImageNet 2012
- Based on optimal batch sizes

HUAWEI

Vision and Value



Contents

Installing MindSpore

Environment Requirements

System Requirements and Software Dependencies

Version	Operating System	Executable File Installation Dependencies	Source Code Compilation and Installation Dependencies
MindInsight 0.2.0-alpha	- Ubuntu 16.04 or later x86_64 - EulerOS 2.8 arrch64 - EulerOS 2.5 x86_64	- Python 3.7.5 - MindSpore 0.2.0-alpha - For details about other dependency items, see requirements.txt.	**Compilation dependencies:** - Python 3.7.5 - CMake >= 3.14.1 - GCC 7.3.0 - node.js >= 10.19.0 - wheel >= 0.32.0 - pybind11 >= 2.4.3 **Installation dependencies:** same as the executable file installation dependencies.

- When the network is connected, dependency items in the requirements.txt file are automatically downloaded during .whl package installation. In other cases, you need to manually install dependency items.

Installation Guide

Installing Using Executable Files

1. Download the .whl package from the MindSpore website. It is recommended to perform SHA-256 integrity verification first and run the following command to install MindInsight:

```
pip install mindinsight-{version}-cp37-cp37m-linux_{arch}.whl
```

2. Run the following command. If `web address: http://127.0.0.1:8080` is displayed, the installation is successful.

```
mindinsight start
```

Method 1: source code compilation and installation

Two installation environments: Ascend and CPU

```
adding 'mindspore/transforms/validators.py'
adding 'mindspore-0.1.0.dist-info/METADATA'
adding 'mindspore-0.1.0.dist-info/WHEEL'
adding 'mindspore-0.1.0.dist-info/top_level.txt'
adding 'mindspore-0.1.0.dist-info/RECORD'
removing build/bdist.linux-x86_64/wheel
------Successfully created mindspore package------
---------------- mindspore: build test end ----------------
```

Method 2: direct installation using the installation package

Two installation environments: Ascend and CPU

Installation commands:

1. pip install –y mindspore-cpu
2. pip install –y mindspore-d

Getting Started

- In MindSpore, data is stored in tensors.

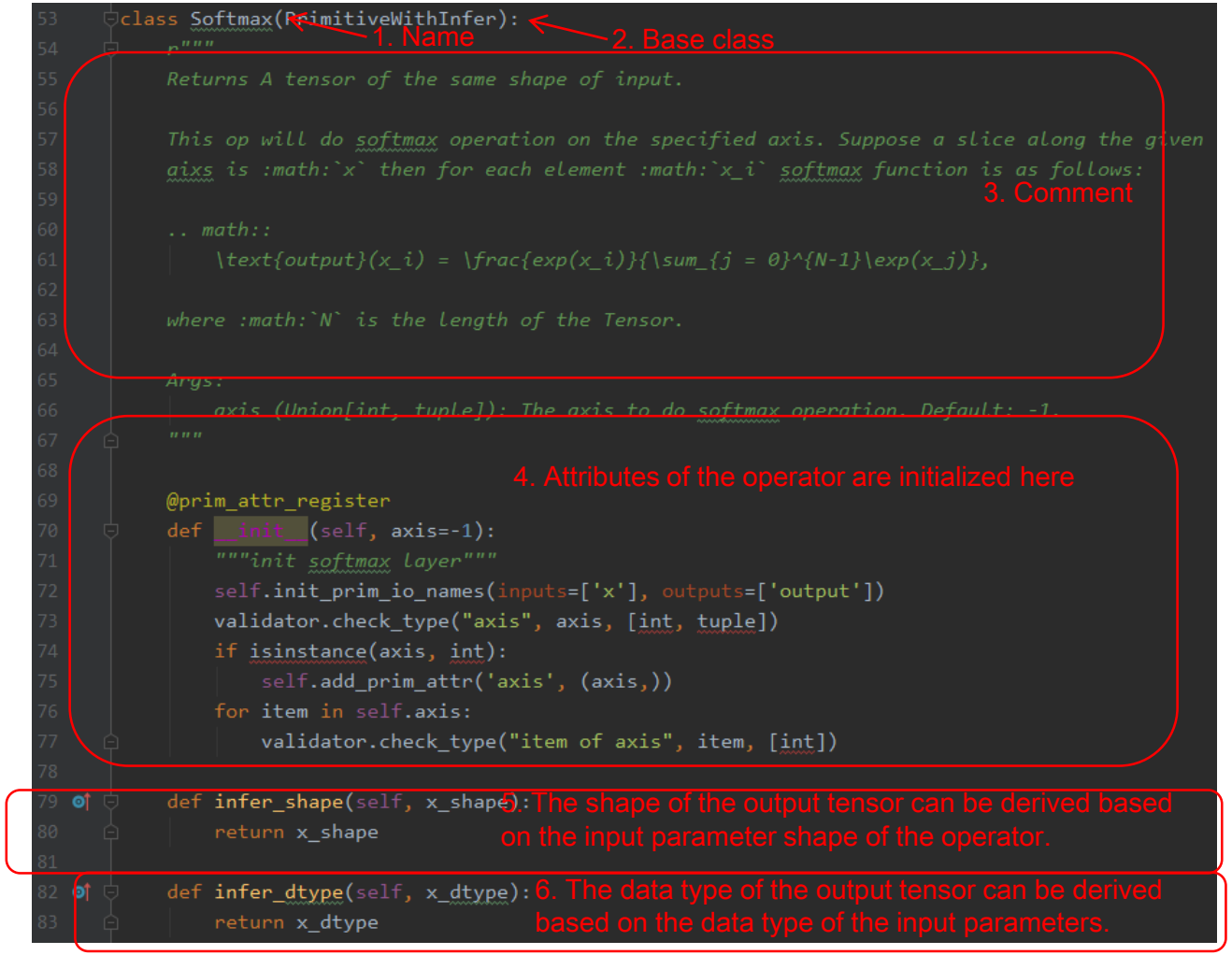
  Common tensor operations:
  - asnumpy()
  - size()
  - dim()
  - dtype()
  - set_dtype()
  - tensor_add(other: Tensor)
  - tensor_mul(other: Tensor)
  - shape()
  - __Str__# (conversion into strings)

Components of ME

Module	Description
model_zoo	Defines common network models
communication	Data loading module, which defines the dataloader and dataset and processes data such as images and texts.
dataset	Dataset processing module, which reads and pro-processes data.
common	Defines tensor, parameter, dtype, and initializer.
context	Defines the context class and sets model running parameters, such as graph and PyNative switching modes.
akg	Automatic differential and custom operator library.
nn	Defines MindSpore cells (neural network units), loss functions, and optimizers.
ops	Defines basic operators and registers reverse operators.
train	Training model and summary function modules.
utils	Utilities, which verify parameters. This parameter is used in the framework.

Programming Concept: Operation

Softmax operator




```
class Softmax(PrimitiveWithInfer):
    """
    Returns A tensor of the same shape of input.

    This op will do softmax operation on the specified axis. Suppose a slice along the given
    aixs is :math:`x` then for each element :math:`x_i` softmax function is as follows:

    .. math::
        \text{output}(x_i) = \frac{exp(x_i)}{\sum_{j = 0}^{N-1}\exp(x_j)},

    where :math:`N` is the length of the Tensor.

    Args:
        axis (Union[int, tuple]): The axis to do softmax operation. Default: -1.
    """

    @prim_attr_register
    def __init__(self, axis=-1):
        """init softmax layer"""
        self.init_prim_io_names(inputs=['x'], outputs=['output'])
        validator.check_type("axis", axis, [int, tuple])
        if isinstance(axis, int):
            self.add_prim_attr('axis', (axis,))
        for item in self.axis:
            validator.check_type("item of axis", item, [int])

    def infer_shape(self, x_shape):
        return x_shape

    def infer_dtype(self, x_dtype):
        return x_dtype
```

Common operations in MindSpore:

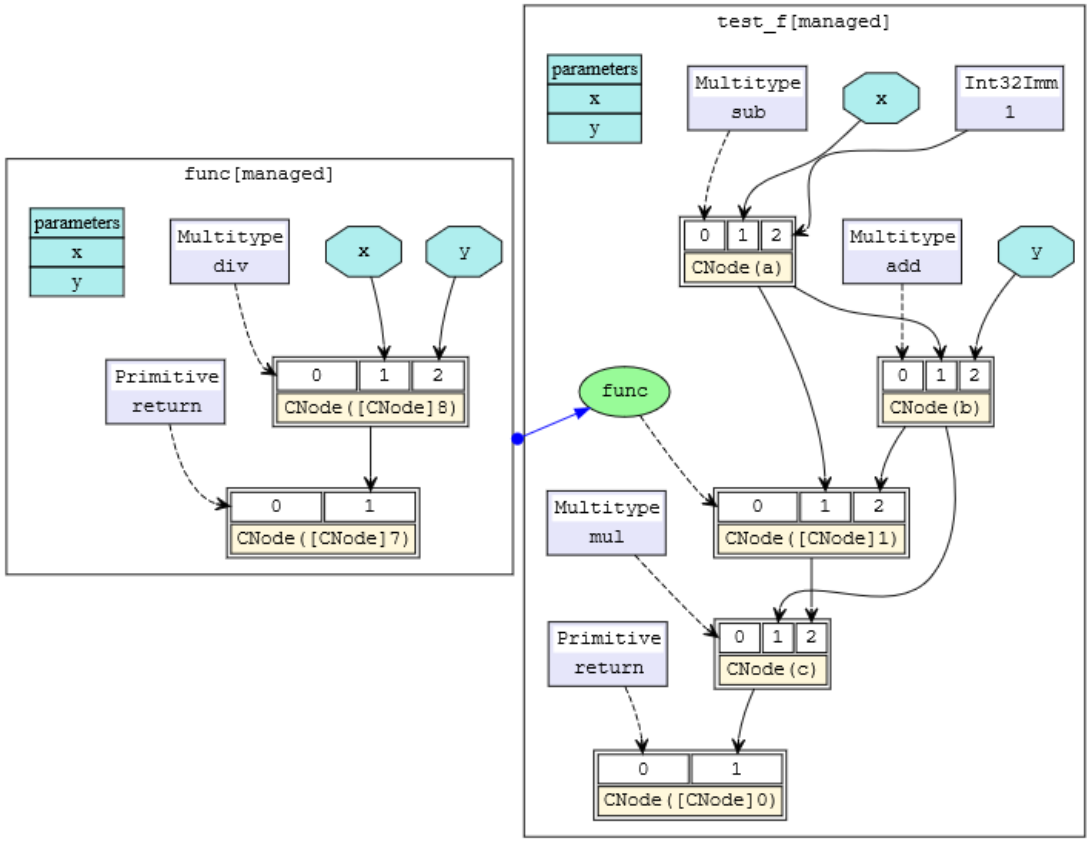
- array: Array-related operators
  - ExpandDims
  - Concat
  - Select
  - ScatterNd…
  - Squeeze
  - OnesLike
  - StridedSlice
- math: Math-related operators
  - AddN
  - Sub
  - Mul
  - MatMul
  - RealDiv
  - ReduceMean
  - Cos
  - Sin
  - LogicalAnd
  - LogicalNot
  - Less
  - Greater…
- nn: Network operators
  - Conv2D
  - Flatten
  - Softmax
  - ReLU
  - Sigmoid
  - Pooling
  - BatchNorm
  - MaxPool
  - AvgPool
  - TopK
  - SoftmaxCrossEntropy
  - SmoothL1Loss
  - SGD
  - SigmoidCrossEntropy…
- control: Control operators
  - ControlDepend
- random: Random operators

Programming Concept: Cell

- A cell defines the basic module for calculation. The objects of the cell can be directly executed.
  - __init__: It initializes and verifies modules such as parameters, cells, and primitives.
  - Construct: It defines the execution process. In graph mode, a graph is compiled for execution and is subject to specific syntax restrictions.
  - bprop (optional): It is the reverse direction of customized modules. If this function is undefined, automatic differential is used to calculate the reverse of the construct part.
- Cells predefined in MindSpore mainly include: common loss (Softmax Cross Entropy With Logits and MSELoss), common optimizers (Momentum, SGD, and Adam), and common network packaging functions, such as TrainOneStepCell network gradient calculation and update, and WithGradCell gradient calculation.

Programming Concept: MindSporeIR

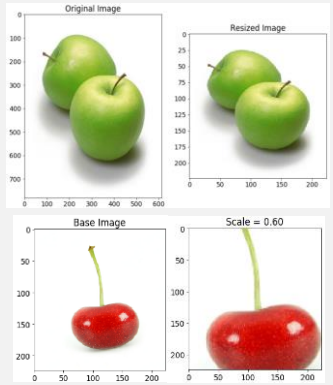
- MindSporeIR is a compact, efficient, and flexible graph-based functional IR that can represent functional semantics such as free variables, high-order functions, and recursion. It is a program carrier in the process of AD and compilation optimization.
- Each graph represents a function definition graph and consists of ParameterNode, ValueNode, and ComplexNode (CNode).
- The figure shows the def-use relationship.



Development Case

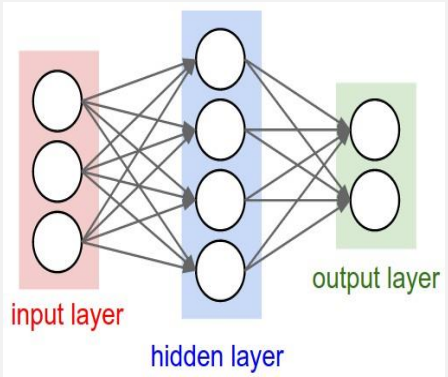
- Let's take the recognition of MNIST handwritten digits as an example to demonstrate the modeling process in MindSpore.

Data



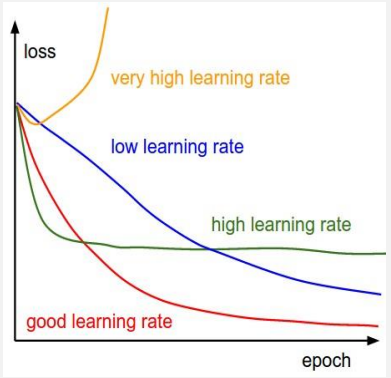
- 1. Data loading
- 2. Data enhancement

Network



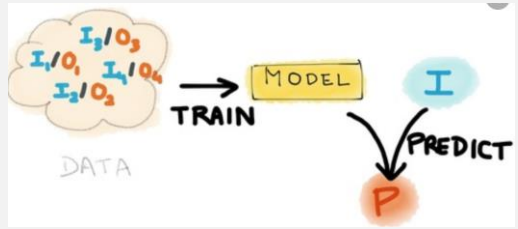
- 3. Network definition
- 4. Weight initialization
- 5. Network execution

Model



- 6. Loss function
- 7. Optimizer
- 8. Training iteration
- 9. Model evaluation

Application



- 10. Model saving
- 11. Load prediction
- 12. Fine tuning

HUAWEI

1. Development Framework

2. **Development and Application**

   - Environment Setup
   - Application Development Cases

Installing MindSpore

Environment Requirements

System Requirements and Software Dependencies

Version	Operating System	Executable File Installation Dependencies	Source Code Compilation and Installation Dependencies
MindInsight 0.2.0-alpha	- Ubuntu 16.04 or later x86_64 - EulerOS 2.8 arrch64 - EulerOS 2.5 x86_64	- Python 3.7.5 - MindSpore 0.2.0-alpha - For details about other dependency items, see requirements.txt.	**Compilation dependencies:** - Python 3.7.5 - CMake >= 3.14.1 - GCC 7.3.0 - node.js >= 10.19.0 - wheel >= 0.32.0 - pybind11 >= 2.4.3 **Installation dependencies:** same as the executable file installation dependencies.

- When the network is connected, dependency items in the requirements.txt file are automatically downloaded during .whl package installation. In other cases, you need to manually install dependency items.

Installation Guide

Installing Using Executable Files

1. Download the .whl package from the MindSpore website. It is recommended to perform SHA-256 integrity verification first and run the following command to install MindInsight:

```
pip install mindinsight-{version}-cp37-cp37m-linux_{arch}.whl
```

2. Run the following command. If `web address: http://127.0.0.1:8080` is displayed, the installation is successful.

```
mindinsight start
```

Method 1: source code compilation and installation

Two installation environments: Ascend and CPU

```
adding 'mindspore/transforms/validators.py'
adding 'mindspore-0.1.0.dist-info/METADATA'
adding 'mindspore-0.1.0.dist-info/WHEEL'
adding 'mindspore-0.1.0.dist-info/top_level.txt'
adding 'mindspore-0.1.0.dist-info/RECORD'
removing build/bdist.linux-x86_64/wheel
------Successfully created mindspore package------
--------------- mindspore: build test end ----------------
```

Method 2: direct installation using the installation package

Two installation environments: Ascend and CPU

Installation commands:

1. pip install –y mindspore-cpu
2. pip install –y mindspore-d

Code example

```python
import os

import mindspore as ms
import mindspore.context as context
import mindspore.dataset.transforms.c_transforms as C
import mindspore.dataset.transforms.vision.c_transforms as CV

from mindspore import nn
from mindspore.train import Model
from mindspore.train.callback import LossMonitor
```

① Import standard libraries and MindSpore。

```python
context.set_context(mode=context.GRAPH_MODE, device_target='Ascend')
```

② Set context：GRAPH mode、Ascend target.

```python
def create_dataset(data_dir, training=True, batch_size=32, resize=(32, 32), rescale=1/(255*0.3081), shift=-0.1307/0.3081, buffer_size=64):
    data_train = os.path.join(data_dir, 'train')
    data_test = os.path.join(data_dir, 'test')
    ds = ms.dataset.MnistDataset(data_train if training else data_test)

    ds = ds.map(input_columns=["image"], operations=[CV.Resize(resize), CV.Rescale(rescale, shift), CV.HWC2CHW()])
    ds = ds.map(input_columns=["label"], operations=C.TypeCast(ms.int32))
    ds = ds.shuffle(buffer_size=buffer_size).batch(batch_size, drop_remainder=True)

    return ds
```

② Data processing：dataset loading and preprocessing

```python
class LeNet5(nn.Cell):
    def __init__(self):
        super(LeNet5, self).__init__()
        self.conv1 = nn.Conv2d(1, 6, 5, stride=1, pad_mode='valid')
        self.conv2 = nn.Conv2d(6, 16, 5, stride=1, pad_mode='valid')
        self.relu = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.flatten = nn.Flatten()
        self.fc1 = nn.Dense(400, 120)
        self.fc2 = nn.Dense(120, 84)
        self.fc3 = nn.Dense(84, 10)

    def construct(self, x):
        x = self.relu(self.conv1(x))
        x = self.pool(x)
        x = self.relu(self.conv2(x))
        x = self.pool(x)
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.fc2(x)
        x = self.fc3(x)

        return x
```

② Model definition：operator initialization, network construction.

```python
def train(data_dir, lr=0.01, momentum=0.9, num_epochs=3):
    ds_train = create_dataset(data_dir)
    ds_eval = create_dataset(data_dir, training=False)

    net = LeNet5()
    loss = nn.loss.SoftmaxCrossEntropyWithLogits(is_grad=False, sparse=True, reduction='mean')
    opt = nn.Momentum(net.trainable_params(), lr, momentum)
    loss_cb = LossMonitor(per_print_times=ds_train.get_dataset_size())

    model = Model(net, loss, opt, metrics={'acc', 'loss'})
    model.train(num_epochs, ds_train, callbacks=[loss_cb], dataset_sink_mode=False)
    metrics = model.eval(ds_eval, dataset_sink_mode=False)
    print('Metrics:', metrics)
```

② Model training and evaluation：Instantiate net, loss, optimizer, loss_monitor

HUAWEI

Dataset loading

Dataset structure	description

ImageFolderDatasetV2	# # Example: cat/image1.jpg, dog/image2.jpg
	ds = ds.ImageFolderDatasetV2(DATA_DIR, class_indexing={"cat":0,"dog":1})

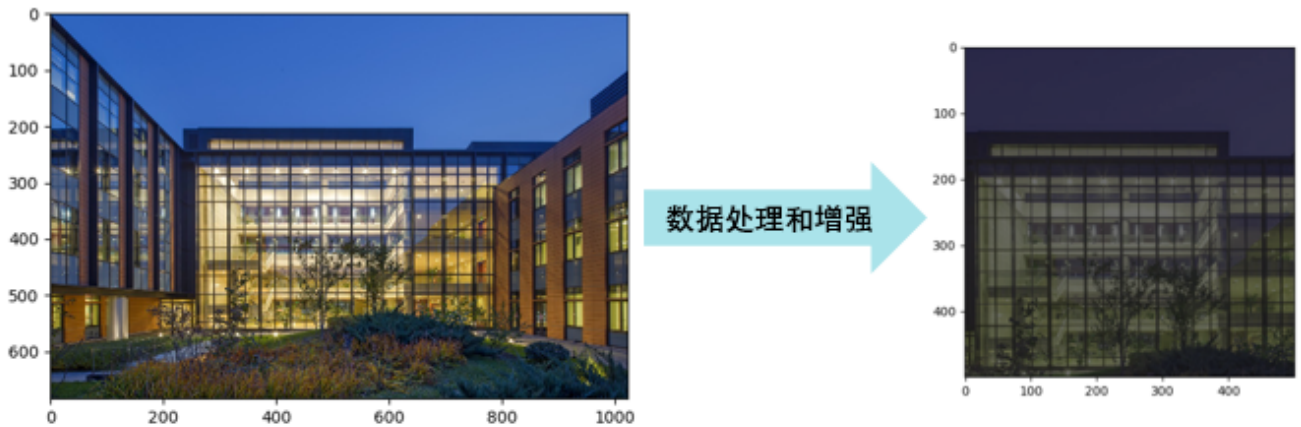
MnistDataset	ds = ms.dataset.MnistDataset(DATA_DIR)
Cifar10Dataset	ds = ms.dataset.Cifar10Dataset(DATA_DIR)
CocoDataset	ds = ds.CocoDataset(DATA_DIR, annotation_file=annotation_file, task='Detection')
MindRecord	ds = ds.MindDataset(FILE_NAME)
GeneratorDataset	ds = ds.GeneratorDataset(GENERATOR, [“image", “label"])

TextFileDataset	dataset_files = ["/path/to/1", "/path/to/2"] # contains 1 or multiple text files
	ds = ds.TextFileDataset(dataset_files=dataset_files)

GraphData	ds = ds.GraphData(DATA_DIR, 2) nodes = ds.get_all_edges(0)

Data processing

- Data processing
  > crop, normalize, rescale, etc.
- Data augmentation
  > Create new data based on existing data to prevent overfitting.



Format conversion: MindSpore supports channel first

NCHW:

NHWC:

```
operations = [
    CV.Resize(resize),
    CV.Rescale(rescale, shift),
    CV.HWC2CHW()
]
ds = ds.map(input_columns=["image"],
operations=operations)
ds =
ds.shuffle(buffer_size=buffer_size).batch(batch
_size, drop_remainder=True)
```

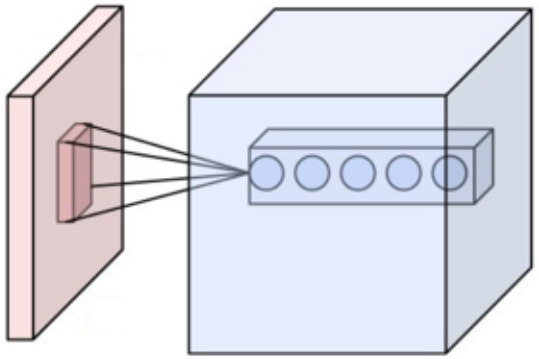
Code example

```
def create_dataset(data_dir, training=True, batch_size=32, resize=(32, 32),
                   rescale=1/(255*0.3081), shift=-0.1307/0.3081, buffer_size=64):
    # data path
    data_train = os.path.join(data_dir,  'train' )
    data_test = os.path.join(data_dir,  'test' )
    # load mnist dataset
    ds = ms.dataset.MnistDataset(data_train if training else data_test)


    # Resize
    # Rescale
    # HWC2CHW
    ds = ds.map(input_columns=[ "image" ], operations=[CV.Resize(resize), CV.Rescale(rescale, shift),
CV.HWC2CHW()])
    # cast to int32
    ds = ds.map(input_columns=[ "label" ], operations=C.TypeCast(ms.int32))
    # when executed on Ascend, set`dataset_sink_mode=True`
    ds = ds.shuffle(buffer_size=buffer_size).batch(batch_size, drop_remainder=True)

    return ds
```

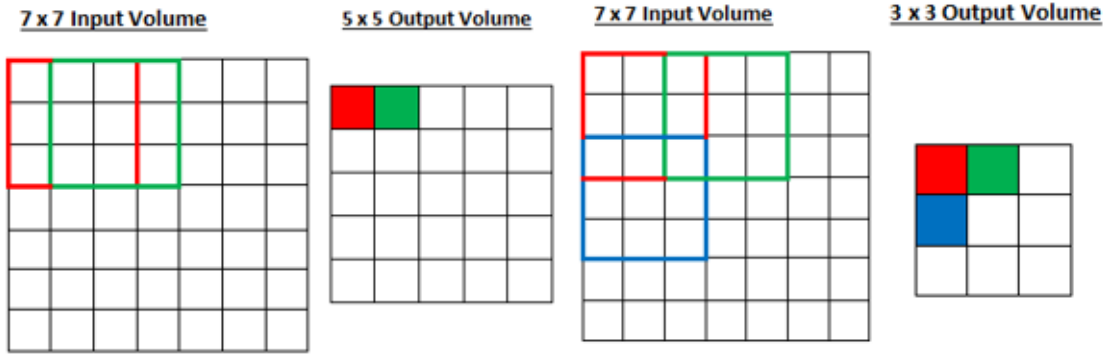
Convolution

```
conv1 = nn.Conv2d(in_channels, out_channels,  ①
                  ②kernel_size=3, stride=1, has_bias=False, weight_init='normal', bias_init='zeros',
                  ③pad_mode='same', padding=0)
```

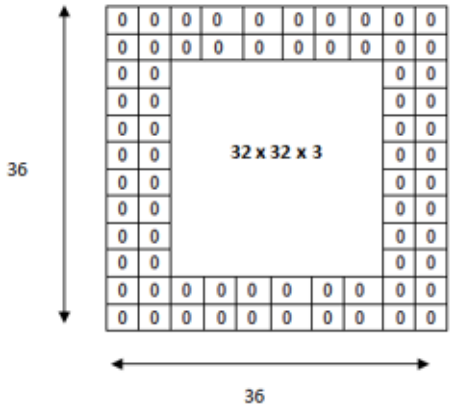
① Input and Output Channels



② Kernel Size and Stride



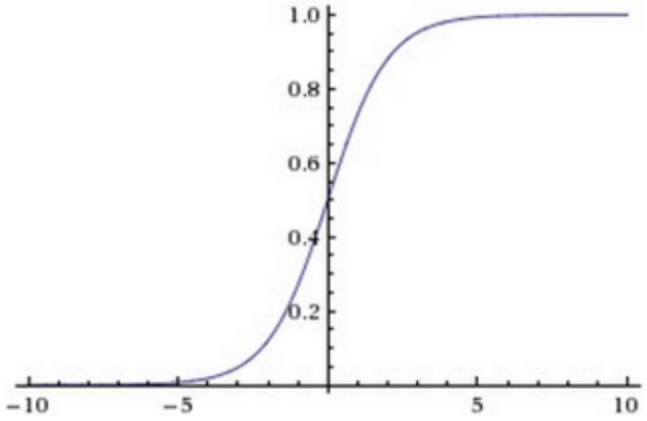
③ Padding



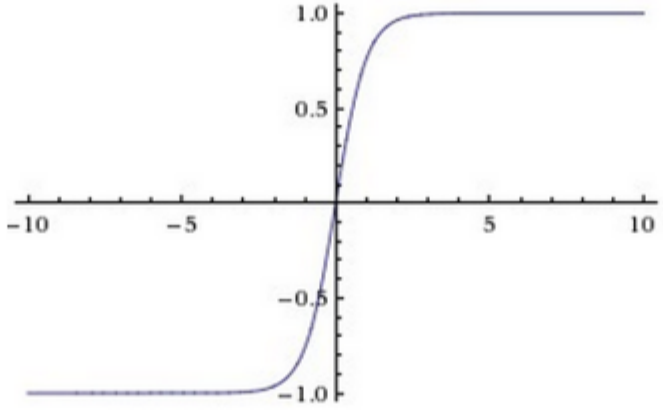
y = conv1(x)
Take a 4D tensor as input and return a 4D tensor as output. batch_size x channels x height x width

Activation function

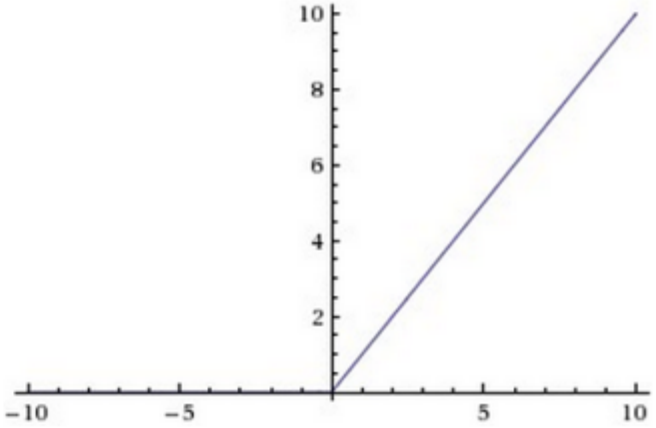
```
relu = nn.Sigmoid()
relu = nn.Tanh()
relu = nn.ReLU()
```



$$Sigmoid(x) = \frac{1}{1+e^{-x}}$$



$$Tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$$

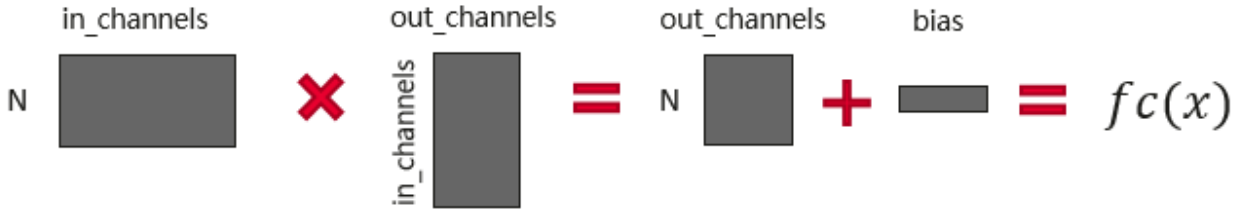


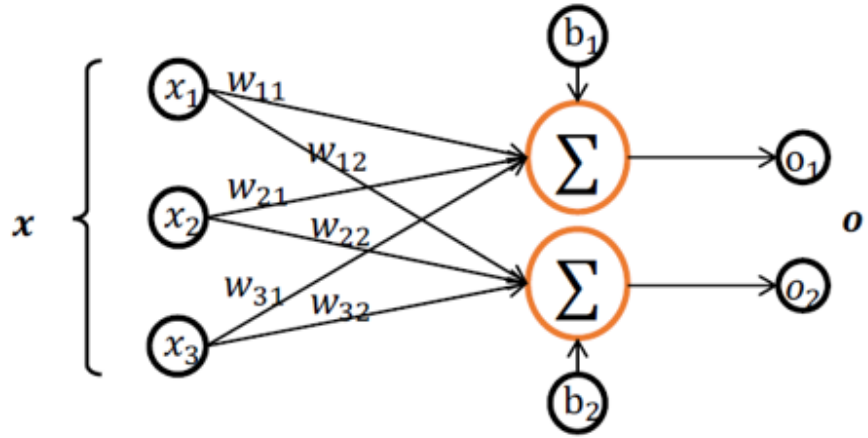
$$ReLU(x) = \max(0, x)$$

Add activation function after conv/fc layer to improve model’ s fitting ability

Dense/Fully connection

```
fc1 = nn.Dense(in_channels, out_channels, weight_init='normal', bias_init='zeros', has_bias=True,
activation=None)
```





y = fc1(x):

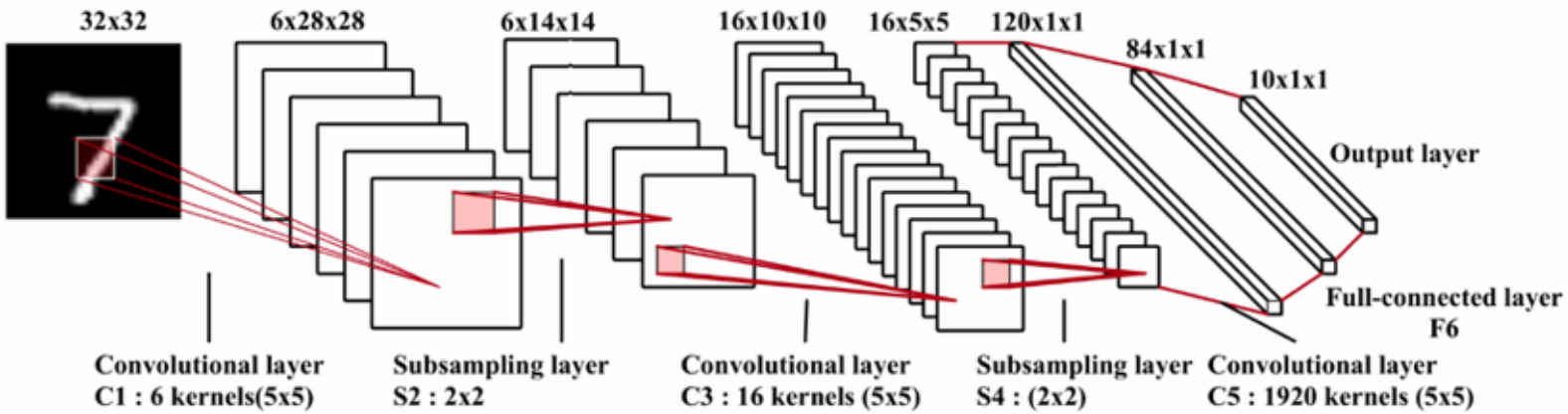
- input: Tensor shape(batch_size, in_channels).
- output: Tensor shape(batch_size, out_channels).

Network definition

```
class LeNet5(nn.Cell):①
    def __init__(self):
        super(LeNet5, self).__init__()
        self.conv1 = nn.Conv2d(1, 6, 5, stride=1, pad_mode='valid')
        self.conv2 = nn.Conv2d(6, 16, 5, stride=1, pad_mode='valid')
        self.relu = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.flatten = nn.Flatten()
        self.fc1 = nn.Dense(400, 120)
        self.fc2 = nn.Dense(120, 84)
        self.fc3 = nn.Dense(84, 10)

    def construct(self, x):
        x = self.relu(self.conv1(x))
        x = self.pool(x)
        x = self.relu(self.conv2(x))
        x = self.pool(x)
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.fc2(x)
        x = self.fc3(x)

        return x
```

② ③

① The network definition must inherit the base class nn.Cell;
② __init__() is based on python interpreter;
③ construct() will be taken over by MindSpore;



(a) LeNet-5 network

Loss function

```
loss = nn.loss.SoftmaxCrossEntropyWithLogits(is_grad=False, sparse=True, reduction='mean')
```

$$Z_i = W \cdot X_i + b$$

$$P_i = \frac{e^{Z_i}}{\sum_j e^{Z_i}}$$

$$l(Z_i, Y_i) = -\log(P_{iY_i})$$

Loss function can measure the difference between predictions and ground truth guiding the update direction of the model. MindSpore now supports：L1Loss、MSELoss、SoftmaxCrossEntropyWithLogits、CosineEmbeddingLoss etc。

Parameter explanation：

- is_grad，whether to calculate only the gradient，default: True；
- sparse，default: False，when it’s True, one_hot operation will be executed on the data；
- reduction，mean、sum。

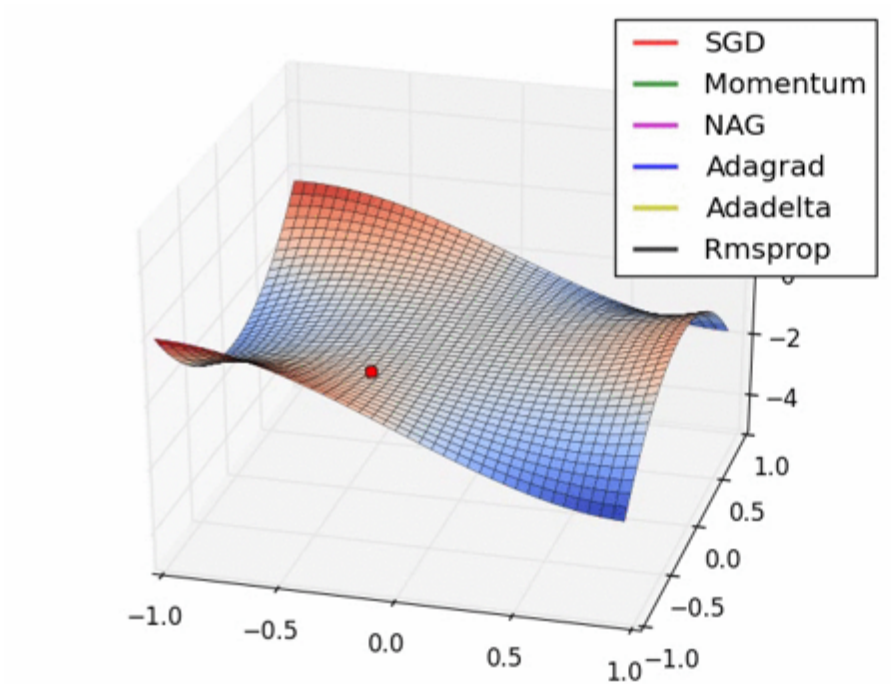
Optimizer

```
opt = nn.Momentum(net.trainable_params(), lr, momentum)
```

- With optimization object（Loss）， we still need optimization strategy, namely optimizer
- MindSpore now supports：SGD、Momentum、RMSProp、Adam、AdamWeightDecay、LARS、FTRL、ProximalAdagrad etc。

SGD with Momentum:

$$v_{t+1} \leftarrow \rho v_t + \nabla_\theta \mathcal{L}(\theta)$$
$$\theta_j \leftarrow \theta_j - \epsilon v_{t+1}$$

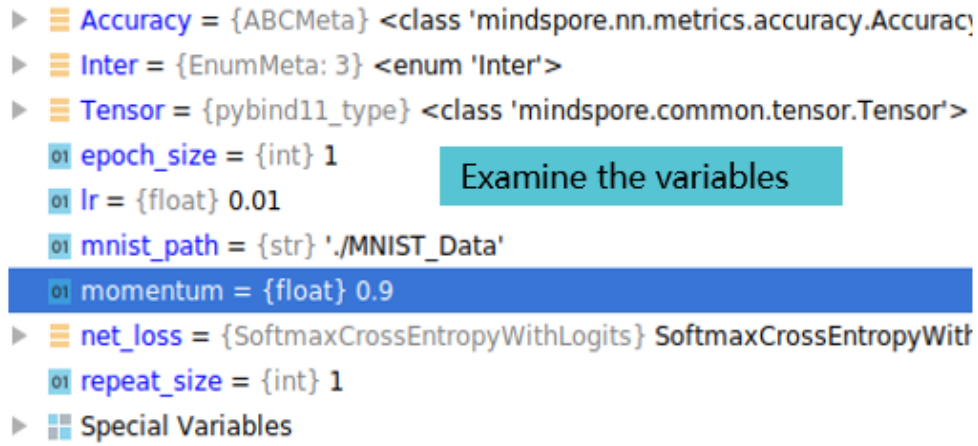


Code debugging

MindSpore supports Graph and PyNative mode, **PYNATIVE mode is** easy to debug, Support Python native debugging method。

```
    net_loss = SoftmaxCrossEntropyWithLogits(is_grad=False, sparse=True, reduction='
    repeat_size = epoch_size  repeat_size: 1
    # create the network
    network = LeNet5()
    # define the optimizer
    net_opt = nn.Momentum(network.trainable_params(), lr, momentum)
    config_ck = CheckpointConfig(save_checkpoint_steps=1875, keep_checkpoint_max=10)
    # save the network model and parameters for subsequence fine-tuning
    ckpoint_cb = ModelCheckpoint(prefix="checkpoint_lenet", config=config_ck)
    # group layers into an object with training and evaluation features
    model = Model(network, net_loss, net_opt, metrics={"Accuracy": Accuracy()})
```

Support native Python debug, set break point



Examine the variables

```
def construct(self, x):
    x = self.relu(self.conv1(x))
    x = self.pool(x)
    print(x.shape)
    x = self.relu(self.conv2(x))
    x = self.pool(x)
    x = self.flatten(x)
    x = self.fc1(x)
    x = self.fc2(x)
    x = self.fc3(x)

    return x
```

Print whatever you want

GRAPH mode possesses high speed performance

```
self.print = P.Print()
self.print(x)
Tensor shape:[[const vector][2, 1]]Int32
val:[[1] [1]]
```

Model Training

- mindspore.train.callback can be used to perform specific tasks during training/evaluation。
  Common Callback：
  > LossMonitor：print loss every step；
  > SummaryStep：collect summary files for visualization。
  > ModelCheckpoint：save checkpoint files；

```
loss_cb = LossMonitor(per_print_times=ds_train.get_dataset_size())
# loss print
epoch: 2 step 1875, loss is 0.4737345278263092
```

- mindspore.nn provides many evaluation metrics such as accuracy、loss、precision、recall、F1。
  > Define a metrics dictionary/set，pass it to your model；
  > Use model.eval to calculate these metrics。
  > model.eval returns a dictionary including corresponding values。

```
metrics={'acc', 'loss'}
# output
{'loss': 0.10531254443608654, 'acc': 0.9701522435897436}
```

Model Evaluation

- mindspore.train.Model provides high level interfaces for training and evaluation.
  - > Input net、loss and opt and complete model initialization;
  - > Use model.train for training，specify the number of iterations（num_epochs）、dataset、callback;

```
def train(data_dir, lr=0.01, momentum=0.9, num_epochs=3):
    ds_train = create_dataset(data_dir)
    ds_eval = create_dataset(data_dir, training=False)

    net = LeNet5()
    loss = nn.loss.SoftmaxCrossEntropyWithLogits(is_grad=False, sparse=True, reduction='mean')
    opt = nn.Momentum(net.trainable_params(), lr, momentum)
    loss_cb = LossMonitor(per_print_times=ds_train.get_dataset_size())

    model = Model(net, loss, opt, metrics={'acc', 'loss'})
    # dataset_sink_mode can be True when using Ascend
    model.train(num_epochs, ds_train, callbacks=[loss_cb], dataset_sink_mode=False)
    metrics = model.eval(ds_eval, dataset_sink_mode=False)
    print('Metrics:', metrics)
```

LeNet5 tutorial：https://gitee.com/mindspore/course/tree/master/checkpoint

Checkpoint

- ModelCheckpoint generates（.meta）and Chekpoint（.ckpt）files every certain steps as you assign.

```
ckpt_cfg = CheckpointConfig(save_checkpoint_steps=steps_per_epoch, keep_checkpoint_max=5)
ckpt_cb = ModelCheckpoint(prefix=ckpt_name, directory='ckpt', config=ckpt_cfg)
model.train(num_epochs, ds_train, callbacks=[ckpt_cb, loss_cb], dataset_sink_mode=dataset_sink)
# 输出:
```

- Use mindspore.train.serialization.load_checkpoint、load_param_into_net to load ckpt files into your net。

```
CKPT_1 = 'ckpt/lenet-2_1875.ckpt '
# load checkpoint; return a dictionary
param_dict = load_checkpoint(CKPT_1)
# load parameters
load_param_into_net(net, param_dict)
load_param_into_net(opt, param_dict)
```

Checkpoint tutorial：https://gitee.com/mindspore/course/tree/master/checkpoint

Model inferencing

- Use MindSpore for inferencing
  - > Load Checkpoint into your net
  - > Call model.predict for inferencing

```
CKPT_2 = 'ckpt/lenet_1-2_1875.ckpt'
def infer(data_dir):
    ds = create_dataset(data_dir, training=False).create_dict_iterator()
    data = ds.get_next()
    images = data['image']
    labels = data['label']
    net = LeNet5()
    load_checkpoint(CKPT_2, net=net)
    model = Model(net)
    output = model.predict(Tensor(data['image']))
    preds = np.argmax(output.asnumpy(), axis=1)
```

- Use other platforms for inferencing
  - > Load Checkpoint into your net
  - > Export your model to a supportive format such as ONNX;
  - > Run you model on any platforms that support ONNX;

```
input = np.random.uniform(0.0, 1.0, size = [1, 3, 224, 224]).astype(np.float32)
export(resnet, Tensor(input), file_name = 'resnet50-2_32.pb', file_format = 'ONNX')
```

MindSpore serving

- MindSpore Serving is a lightweight, high-performance service module. It is designed to help developers efficiently deploy online services.
  - > Use MindSpore to train your model
  - > Export MindSpore model（MINDIR）;
  - > Use MindSpore Serving to create inference service。

```
ms_serving [--help] [--model_path <MODEL_PATH>] [--model_name <MODEL_NAME>]
           [--port <PORT>] [--device_id <DEVICE_ID>]
```

parameter	attribute	description	Parameter type	Default	ranges
--help	optional	Display commands help。	-	-	-
--model_path <MODEL_PATH>	required	Set model path。	str	空	-
--model_name <MODEL_NAME>	required	Set model name。	str	空	-
--port <PORT>	optional	Set service port number。	int	5500	1~65535
--device_id <DEVICE_ID>	optional	Set Device ID	int	0	0~7

https://www.mindspore.cn/tutorial/zh-CN/r0.6/advanced_use/serving.html

Interfaces comparison

MindSpore shares the similar API and code style as tf.keras and pytorch， easy to use and migrate

```
class Net(tf.keras.Model):
  def __init__(self):
    super(MyModel, self).__init__()
    self.conv1 = tf.keras.layers.Conv2D()
    self.relu = tf.keras.layers.ReLU()
    self.fc1 = tf.keras.layers.Dense()

  def call(self, x):
    x = self.conv1(x)
    x = self.relu(x)
    x = self.fc1(x)
    return x

model = Net()
loss_fn =
tf.keras.losses.SparseCategoricalCrossentropy()
opt = tf.keras.optimizers.Adam()

data = tf.keras.datasets.cifar10.load_data()

model.compile(optimizer=opt, loss=loss_fn,
metrics=['accuracy'])
model.fit(data[0], data[1], 5)
model.evaluate(data[0], data[1])
```

Tensorflow.keras

```
class Net(nn.Cell):
  def __init__(self):
    super(Net, self).__init__()
    self.conv1 = nn.Conv2d()
    self.relu = nn.ReLU()
    self.fc1 = nn.Dense()

  def construct(self, x):
    x = self.conv1(x)
    x = self.relu(x)
    x = self.fc1(x)
    return x

net = Net()
loss_fn =
nn.loss.SoftmaxCrossEntropyWithLogits()
opt = nn.Momentum()

dataset = dataset.Cifar10Dataset()

model = train.Model(net, loss_fn, opt,
metrics={'acc'})
model.train(5, dataset)
model.eval(dataset)
```

MindSpore

```
class Net(nn.Module):
  def __init__(self):
    super(Net, self).__init__()
    self.conv1 = nn.Conv2d()
    self.fc1 = nn.Linear()

  def forward(self, x):
    x = self.conv1(x)
    x = F.relu(x)
    x = self.fc1(x)
    return x

net = Net()
loss_fn = nn.CrossEntropyLoss()
opt = optim.SGD(momentum=0.9)

dataset = torchvision.datasets.CIFAR10()
loader = torch.utils.data.DataLoader(dataset)

for epoch in range(5):
  for i, data in enumerate(loader, 0):
    opt.zero_grad()
    outputs = net(data[0])
    loss = loss_fn(outputs, data[1])
    loss.backward()
    opt.step()

with torch.no_grad():
  for data in dataloader:
    outputs = net(data[0])
    _, pred = torch.max(outputs.data, 1)
    total += data[1].size(0)
    correct += (pred == labels).sum().item()
    acc = 100 * correct / total
```

Pytorch

HUAWEI

Quiz

1. In MindSpore, which of the following is the operation type of nn? ( )

   A. Mathematical

   B. Network

   C. Control

   D. Others

Summary

- This chapter describes the framework, design, features, and the environment setup process and development procedure of MindSpore.

More Information

TensoFlow: https://www.tensorflow.org/

PyTorch: https://pytorch.org/

Mindspore: https://www.mindspore.cn/en

Ascend developer community: http://122.112.148.247/home

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and organization for a fully connected, intelligent world.

HUAWEI